Least Connections and Least Response Time

route by the load a server actually has

Watch the 4-min video

In one line: least connections sends each new request to the backend with the fewest active requests, so slow requests stop piling up; least response time also prefers recently fast backends (measured with a moving average), and power of two choices (pick two at random, take the less loaded) keeps many balancers from stampeding onto one server.

1 The problem round robin can't see

Your servers stream answers from a language model. Some responses finish in 2 seconds, others take a full minute. Round robin keeps handing out requests in turn, and soon one server juggles twenty long streams while another has two. To spread *work*, the balancer has to look at the load each server actually has.

2 Least connections

Least connections tracks how many requests (or connections) each backend is handling right now. Each new request goes to the backend with the **fewest**. When slow requests tie up a server, its count stays high, so new work flows elsewhere. It adapts to variable request durations automatically, without knowing them in advance.

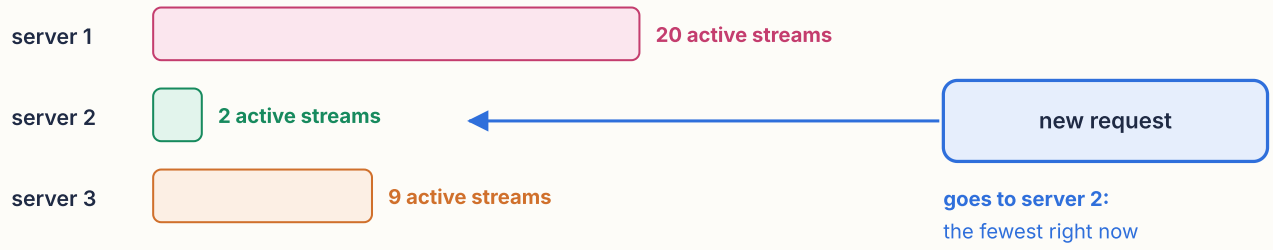


Figure 1. Least connections: the new request goes to the server with the fewest active requests right now, not the next one in turn.

Weighted least connections divides the count by capacity: a server with weight 2 may hold twice as many before it counts as equally busy.

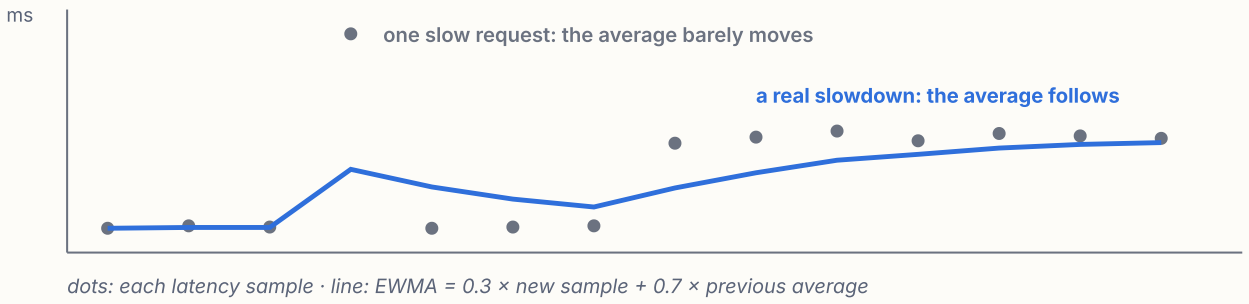
3 Least response time and EWMA

Least response time goes further: it prefers the backend with the lowest *recent latency*, usually combined with its active request count. A server that is slow because of garbage collection, a hot disk or a noisy neighbour gets less traffic until it recovers.

To measure recent latency, balancers keep an **exponentially weighted moving average** (EWMA). Each new sample nudges the average, and old samples fade away:

```
average = α × new sample + (1 − α) × previous average (e.g. α = 0.3)
```

It reacts quickly to a real slowdown but doesn't overreact to one slow request. Envoy, Linkerd and Finagle all offer latency-aware balancing like this.



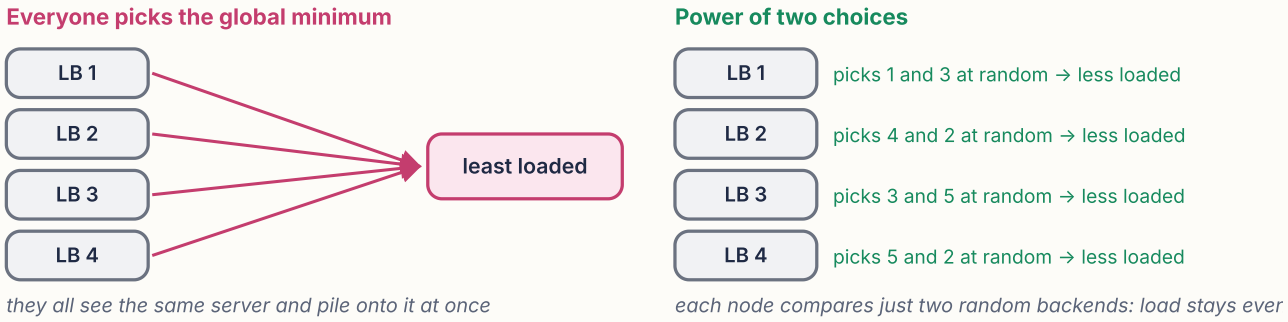
dots: each latency sample · line: EWMA = 0.3 × new sample + 0.7 × previous average

Figure 2. An EWMA of latency samples: a single slow request barely moves it; a lasting slowdown pulls it up within a few samples.

4 The herd problem and power of two choices

Always picking the global minimum has a catch. With many balancer nodes, they all see the same least loaded server, and all send their next requests there at once. It's suddenly the busiest server. Their view of load is also slightly out of date, which makes the stampede worse.

Power of two choices fixes it: pick **two** backends at random, and send the request to the **less loaded** of the two. Surprisingly, this nearly matches a full search. In the maths, the most loaded server goes from about log n / log log n above average with pure random choice to about log log n with two choices: an exponential improvement. It is cheap, needs only local information, and scales to thousands of backends. Envoy's default least-request balancer works this way.



they all see the same server and pile onto it at once

each node compares just two random backends: load stays ever

Figure 3. Left: every balancer chases the same minimum and overloads it. Right: each compares two random backends, so the choices spread out.

WORKED EXAMPLE: AN LLM GATEWAY

A gateway streams tokens from a pool of GPU inference servers. Each stream lasts between 2 seconds and 2 minutes. With least connections, each new chat goes to the server with the fewest active streams, so long generations don't pile up on one GPU. Several gateway nodes run power of two choices, so they don't all pick the same quiet server, and the number of active streams stays within a few of each other across the pool.

TRADE-OFFS

- The balancer must track state per backend (counts, latencies), so it's slightly more work than round robin.
- Counts are local to each balancer node; with many nodes, each sees only its own traffic. That's fine with power of two choices.
- A server that fails fast (instant errors) looks lightly loaded and attracts traffic: combine with health checks and outlier ejection.

5 Key terms

Least connections: pick the backend with the fewest active requests

Least response time: prefer the backend with the lowest recent latency

Herd behaviour: many balancers choosing the same server at once

Outlier ejection: temporarily remove a backend that keeps failing

Weighted least conns: count divided by capacity weight

EWMA: moving average where old samples fade

Power of two choices: pick two at random, use the less loaded

6 Interview questions

Q1. Why is least connections better for variable request duration? BASIC

It routes by current active requests, not by turn, so a server busy with slow requests automatically gets fewer new ones, without knowing request durations in advance.

Q2. What is power of two choices? BASIC

Pick two backends at random and send the request to the less loaded one. It nearly matches a full search for the minimum, needs only local information, and stops every balancer from stampeding to the same server.

Q3. How does least response time measure latency? BASIC

With an exponentially weighted moving average of recent response times, often combined with the active request count, so a server that slows down gets less traffic until it recovers.

Q4. A buggy backend returns 500 errors in 1 ms. Least connections sends it more and more traffic. Why, and how do you fix it? SCENARIO

Fast failures keep its active count low, so it looks idle. Add outlier ejection (remove backends with high error rates for a while) and health checks, so a failing backend leaves the pool.

Q5. You have 20 gateway nodes in front of 200 GPU servers serving long streams. Which algorithm? SCENARIO

Least connections (least requests) with power of two choices on each gateway node: it balances active streams without a global view and without the herd effect of every node choosing the same minimum.

Q6. Why not always pick the server with the absolute lowest load? TRICKY

With many balancers and slightly stale load data, they all pick the same server simultaneously and overload it, then all move to the next. Randomising between two candidates breaks that synchronisation.

Next: SD-G-005 · Consistent Hashing backendroadmap.com